رواد مصر الرقمية

وزارة الاتصالات
وتكنولوجيا المعلومات

Task Management System - Project Planning & Management

Developed By

Ahmed Osama Ali
Marko Emil Bebawy

Task Management System: A dynamic web-based tool for seamless task creation, assignment, tracking, and collaboration. Real-time notifications, user-friendly design, and robust task prioritization capabilities for enhanced productivity.

Project Plan

1. **Project Start & Requirements Analysis**

   20-Feb-2024 to 05-Mar-2024

   2 Weeks

2. **UI/UX Design**

   06-Mar-2024 to 13-Mar-2024

   1 Week

3. **Documentation Submission**

   21-Mar-2024

4. **Frontend Development**

   27-Mar-2024 to 09-May-2024

   6 Weeks

5. **Testing & Debugging**

   10-May-2024 to 31-May-2024

   3 Weeks

6. **Deployment**

   01-Jun-2024 to 07-Jun-2024

   1 Week

7. **Project Submission**

   09-May-2024

- **Milestones:** Requirements gathered | UI/UX finalized | Frontend developed | Testing & debugging | Deployment & go-live.
- **Deliverables:** Requirement Document | UI/UX Wireframes | Functional Prototype | Final System Deployment | Project Report.

Task Assignment & Risk Assessment

Task Assignment & Roles

Team Member	Role Responsibilities
Ahmed Osama Ali	
Marko Emil Bebawy	

Risk Assessment & Mitigation Plan

Risk	Impact	Mitigation Strategy
Scope Creep	High	Define clear project requirements early.
Missed Deadlines	Medium	Regular progress tracking and agile development.
Security Vulnerabilities	High	Implement authentication and data encryption.
UI/UX Complexity	Medium	Conduct usability testing with end-users.

KPIs (Key Performance Indicators)

System Uptime

Aim for 99.9% availability to ensure reliable task management. Consistent uptime is critical for user trust and productivity.

Task Completion Rate

Track the percentage of completed tasks versus assigned tasks. High completion rates show effective task management.

Response Time

Maintain system response times under 2 seconds. Fast response promotes efficient workflow.

User Adoption Rate

Monitor active users within the first 3 months. Widespread adoption means the system meets user needs.

Literature Review

(TBD)

Requirements Gathering

Stakeholder Analysis

Stakeholder Needs Project Manager Overview of tasks and team progress.

Team Members Ability to create, assign, and track tasks.

End Users Simple and intuitive task management system.

User Stories & Use Cases

- **User Story 1:** As a user, I want to create tasks with deadlines so that I can manage my workload efficiently.
- **User Story 2:** As a manager, I want to assign tasks to my team members and track their progress.
- **Use Case:**
User logs in → Creates a new task → Assigns it to a team member → Sets a deadline → Receives notifications on task status.

Functional Requirements

- Task creation, assignment, and categorization.
- Task progress tracking and reporting.
- Task categorization and priority management.
- Task filtering and search functionality.
- Reporting and analytics dashboard for performance.

Non-functional Requirements

- Performance: Fast load time and response under 2 seconds.
- Security: User authentication and role-based access control.
- Usability: Intuitive UI for easy task management.
- Reliability: 99.9% system uptime.

Problem Statement & Objectives

Problem Statement

Lack of Intuitive Tools
Many individuals and teams struggle with task organization due to the absence of user-friendly solutions.

Complex & Inflexible Solutions
Existing tools can be overly complicated or rigid, making it difficult to:

- Track task progress
- Prioritize work
- Filter tasks by status

Objectives

User-Friendly Design
Develop a simple and intuitive task management app using React.

Comprehensive Task Control

Allow users to:

- Create, update, and delete tasks
- Categorize tasks and set priority levels
- Assign due dates

Smart Organization
Implement filtering and search features based on task status (Completed, In-Progress).

Robust Architecture
Utilize Redux for state management and a Node.js/Express API for backend functionality.

Use Cases

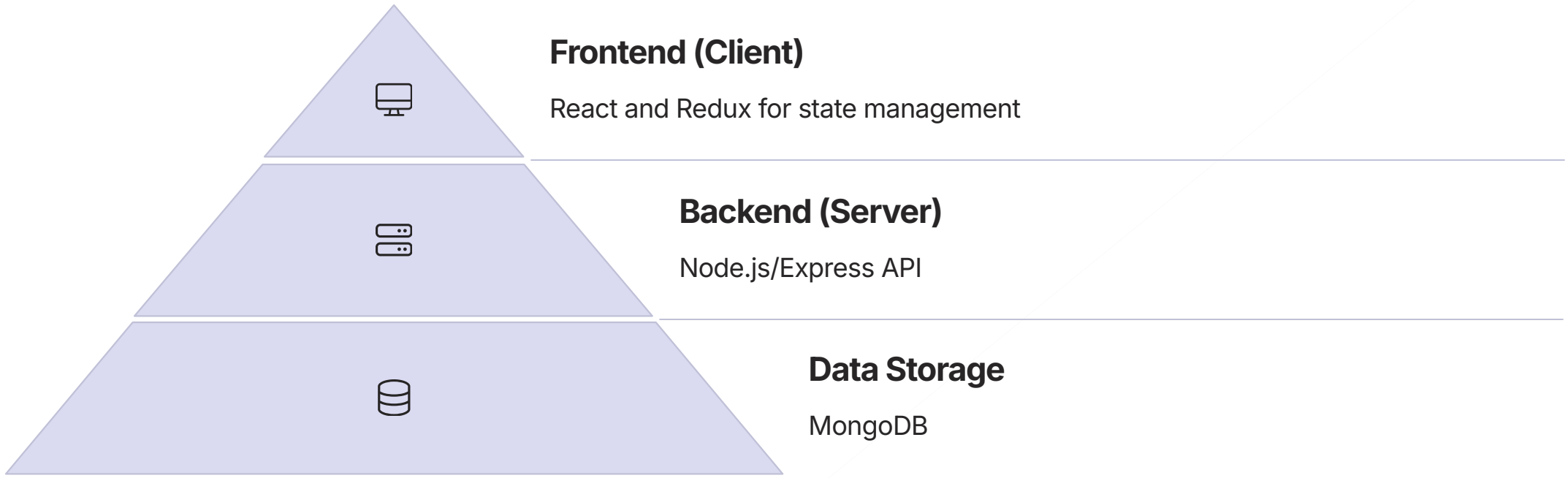
Create Tasks
Easily add new tasks with details like title, priority, and due date.

Update Progress
Edit existing tasks, such as marking them as completed.

Eliminate Clutter
Remove tasks from the system when they're no longer needed.

Filter & Search
Quickly find tasks by status, priority, or keywords in the title/description.

Software Architecture



- **Architecture Style – MVC (Model-View-Controller)**
  Model – Task data managed by **Redux** (frontend) and **backend API**
  View – UI rendered using **React components**
  Controller – **Event handlers** in React and **API routes** in Node.js

- **Database Design – Simple Task Structure**
  Task:
  - **ID** – Unique identifier
  - **Title** – Task name (string)
  - **Priority** – Low, Medium, High
  - **Due Date** – Task deadline (date)
  - **Status** – Completed, In-Progress

Data Flow & System Behavior

User Input

User submits task form

Frontend Processing

React component sends data to Redux

Redux dispatches action to API (POST request)

API Communication

Node.js API processes the request

Data Storage

API saves the task to MongoDB and gets back a unique ID

Response Handling

API returns success response

UI Update

Redux updates state, and UI re-renders with the new task

Activity Diagram (Task Creation):

Start → Enter Task Details → Submit Form → Validate Input → Send to API → Save Task → Display Updated List → End.

State Diagram (Task):

States: In-Progress → Completed.

Transitions: Create (In-Progress), Update (In-Progress ↔️ Completed), Delete (Removed).

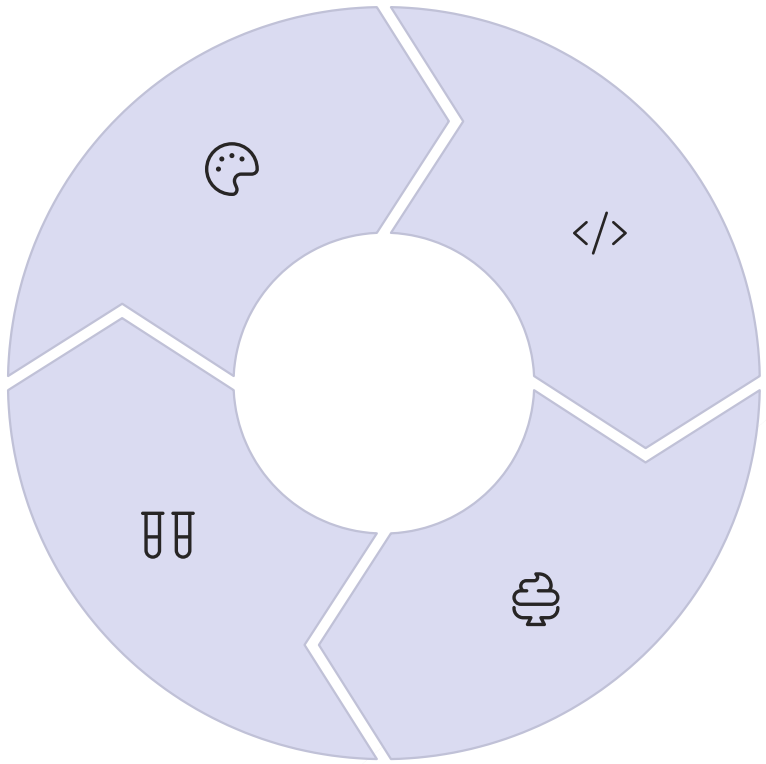
UI/UX Design & Deployment

UI/UX Design

Minimalist, intuitive navigation with blue (primary), white (background), gray (accents)

Testing

Unit Tests, Integration Tests, and User Acceptance Testing



Technology Stack

Frontend: React, Redux, HTML, CSS, JavaScript. Backend: Node.js, Express. Database: MongoDB.

Deployment

Client: Browser running React app → Hosted on Netlify. Server: Node.js/Express API → Hosted on Heroku.

API Documentation:

- **POST /tasks** – Create a new task
  - **Body Parameters**: title, priority, dueDate
- **GET /tasks** – Retrieve all tasks
- **PUT /tasks/:id** – Update an existing task
- **DELETE /tasks/:id** – Remove a task

Component Diagram:

Components: TaskList (React) TaskForm (React)

TaskAPI (Node.js).

Dependencies: TaskList ↔️ Redux ↔️ TaskAPI.